AN ANALYSIS OF DIJKSTRA'S AND A-STAR ALGORITHMS IN TWO DIMENSIONAL GRIDS

COLBY WIRTH

Abstract. A graph is an abstract mathematical structure that consists of a set of vertices (or nodes) and edges connecting pairs of vertices. A graph G is commonly represented as $G = (V, E)$, where V is the set of vertices and E is the set of edges. Many graph-related problems involve the traversing of searching for vertices. For example, GPS navigation is generally built around one key feature: finding the set of shortest paths from a point of origin to a destination. This problem has been solved and optimized with a plethora of algorithms. However, there lacks comprehensive academic analyses comparing the performances of these algorithms across many domain spaces. This paper analyzes Dijkstra's Algorithm and a selection of A-star Algorithms in a domain space similar to GPS navigation. Specifically, the runtimes and traversing efficiencies of these algorithms are explored on three connected, undirected graphs under various conditions, while maintaining the structure of a two dimensional grid-space similar.

Keywords: Graph Theory, Dijkstra's algorithm, A-star algorithm, path finding, graph traversal, computational analysis, heuristics

1. Introduction

The algorithms studied in this paper-Dijkstra's Algorithm and a selection of A-star algorithms—have been extensively developed, optimized, and analyzed. However, there remains a gap in the literature regarding the comparative performance of these algorithms, specifically in terms of runtime and traversal efficiency. This paper aims to fill that gap by analyzing the performance of these algorithms in terms of runtime and the number of nodes traversed. The analysis quantifies the advantages that the A-star algorithms provide over Dijkstra's in this limited domain. This domain space can be expanded in future studies to provide further insights and guide the selection of the most appropriate algorithm for various real-world problems.

2. Literature Review

A Note on Two Problems in Connexion

Published in 1959 by E.W. Dijkstra, this work has been cited over 36,000 times in academic papers as it is recognized as the first formalization of a shortest path algorithm. This is Dijkstra's Algorithm, also known as *Best-First Search* Algorithm. For brevity, the crux of the algorithm is described below in short terms:

2000 *Mathematics Subject Classification.* 68R10, 68-02.

When finding the shortest path between two given nodes P and Q, the existence of a node R on the minimal path implies the existence of a minimal path from P to R. Dijkstra describes the algorithm as the process of iteratively building minimum paths from P to each R in increasing length until Q is found [1]. The equation for calculating the next shortest path on any given iteration is as follows:

$$g[v] = min(g[v],\ g[u] + w[u, v])$$

Where $g[v]$ is the shortest known path from the origin node (o) to node v. $g[u]$ is the distance from the origin node to the current node u, and $w[u, v]$ represents the weight (distance) of the edges between u to v. $g[v]$ is calculated with each iteration of the algorithm, as the minimum paths are built until the goal node is found. The equation above can be simplified to $f(v) = g(v)$.

Initial conditions for $g[o] = 0$, the distance from the origin to itself is 0. The distance to all neighboring[1] unvisited nodes v, is set to ∞ at each iteration [3]. With each iteration, the proper distance is discovered, and the edge value is updated accordingly. Then, the edge with the shortest cumulative distance from the origin node is selected and iterated on; This is the building of the next shortest path. These steps are repeated until the goal node is found: $v =$ goal.

A Formal Basis for the Heuristic Determination of Minimum Cost Paths

Publish in 1968, *A Formal Basis for the Heuristic Determination of Minimum Cost Paths* outlines the framework of heuristics, specifically *admissible* heuristics conceptually and mathematically. Generally, a heuristic conveys special knowledge about a problem domain space. The class of algorithms that utilize the Best-First Search algorithm with the addition of a heuristic are called A-Star (A*) Algorithms. The heuristic, $h(v)$ is the distance from the current node in iteration to the goal node [2]:

$$f(v) = g(v) + h(v)$$

However, the true value of $h(v)$ is unknown until the goal node is found, so an estimated distance, $h^*(v)$, to the goal node is calculated. This is the heuristic function:

$$f^*(v) = g^*(v) + h^*(v)$$

Where $g^*(v)$ is the cost to reach the node at the the current iteration of the algorithm and $f^*(v)$ is the estimated cost to reach the goal node from node n [2].

The admissible class of heuristics is defined as the set heuristic functions that guarantee to find shortest path, predicated on the assumption that the path exists. More formally, a heuristic is admissible if and only if for all nodes, v, an estimated distance is less than or equal to the distance of the shortest path [2]:

$$h(v) \leq h^*(v)$$

[1] Two nodes are neighbors or *adjacent* if they are connected by an edge

The proof can be found on pages 3 and 4 of the referenced paper.

3. Main Content

Experiment Methodology

In total, 72 trials were conducted, 18 per algorithm. Each algorithm was tested with three different classifications of edge connections:

i. A Graph with uniform orthogonal and diagonal distances.

ii. A Graph with standard Octile distances (orthogonal and diagonal lengths of 10 and 14 respectively)

iii. A Graph with randomly generated edge lengths that generally maintain longer diagonal edges over orthogonal edges

Additionally, for each condition, each algorithm was tested on three separate types of graphs: Conceptually, the graphs can visualized as an n x n grid with total node counts of 100 nodes, 10,000 nodes, and 1,000,000 nodes. This allows for the analysis of the algorithms across small, medium, and large sized graphs while maintaining relative characteristics of the graph regarding node density and distribution.

For each combination of all graph sizes and algorithms above, two types of traversals were tested: A diagonal and a horizontal traversal. By having these two two traversal types, all potential ways to traverse between two nodes in this space are executed.

A-Star Family Overview

i.) Manhattan Distance Heuristic
The Manhattan Distance is defined as the distance measured along axes at right angles. This distance is calculated as

$$h(v_i) = |x_1 - x_2| + |y_1 - y_2|$$

Also known as taxi-cab distance, this estimation can be conceptualized as the shortest path between two nodes as traveled along orthogonal axes [4].

ii.) Euclidean Distance Heuristic
The Euclidean Distance is defined as the distance measured along the diagonal between two vertices [4]. It can calculated as :

$$h(v_i) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Also known Straight-line distance or As-the-crow-flies distance, this estimation can be thought of as the shortest direct path on a coordinate plane between two vertices.

iii) Octile (Diagonal) Distance Heuristic

The Octile Distance Heuristic is used to search graphs with orthogonal and diagonal distances of 1 and $\sqrt{2}$ respectively [3]. In this space, these distances are scaled by ten to get 10 and ≈ 14 to simplify the calculations. This is inherently lossy, however the general patterns are still captured with a high degree of precision. The equation can defined as:

$$h(v_i) = 10 \cdot (dx + dy) + (14 - 10) \cdot min(dx, dy), \text{ where}$$
$$dx = abs(x_1 - x_2)$$
$$dy = abs(y_1 - y_2)$$

The Octile distance is an extension of the Manhattan distance, as it includes both orthogonal and diagonal traversals.

Analysis Algorithm Performance

Overview

The averages of **nodes traversed** and **runtimes** across all experiments of each graph size are used for each tabulation. Each data point has been normalized with respect to the performance of Dijkstra's algorithm for the given graph size[2].

Number of Nodes	Dijkstra	A* w/ Euclidean	A* w/ Manhattan	A* w/ Octile
100	1.0	0.93	0.12	0.12
10000	1.0	0.94	0.01	0.01
10000000	1.0	0.91	0.02	0.01

FIGURE 1. Nodes Traversed Normalized w.r.t. Dijkstra

Number of Nodes	Dijkstra	A* w/ Euclidean	A* w/ Manhattan	A* w/ Octile
100	1.0	1.15	0.62	0.48
10000	1.0	1.10	0.15	0.11
10000000	1.0	0.96	0.006	0.004

FIGURE 2. Runtimes Normalized w.r.t. Dijkstra

Dijkstra's Algorithm:

As shown in Figures 1 and 2, Dijkstra's algorithm was the least performant of all algorithms. Because the other algorithms operate with a heuristic and are in essence a optimization of Dijkstra's, this algorithm serves as the benchmark for comparison.

A-Star with Euclidean

The A-star algorithm with Euclidean distance outperforms Dijkstra's in terms of nodes traversed in all cases. It performs 10% and 15% slower in run-time compared to Dijkstra's on graphs with 100 and 10,000 nodes, respectively. However, on graphs with 1,000,000 nodes, the A* algorithm is 4% faster. This suggests that the performance benefit from A* becomes noticeable on graphs with 1,000,000 nodes.

[2] All unaggregated data can be found in the appendix.

In terms of node traversals, A* is consistently more efficient, performing 1.07 to 1.09 times better.

A-star with Manhattan
The A-star algorithm with Manhattan distance outperforms Dijkstra's across all metrics and in all cases. In terms of runtime, A* is 1.62 times faster with graphs of 100 nodes, 6.7 times faster with graphs of 10,000 nodes, and 167 times faster with graphs of 1,000,000 nodes. In terms of nodes traversed, A* is 8.3 times, 100 times, and up to 50 times more efficient across the three graph sizes.

A-Star with Octile
Across all metrics, this algorithm performed the best. In terms of run-time, it was between 2.09 and 250 times faster than Dijkstra's. With nodes traversed, it traversed between 8.3 and 100 times fewer nodes than Dijkstra's.

4. Conclusion

The performances of four graph traversal algorithms were presented and analyzed within the context of a cubical grid-like domain space. In this domain, all nodes could be conceptualized on a two-dimensional grid, with edges connecting diagonally and orthogonally adjacent nodes. This structure constrained the traversal to relatively simple orthogonal or diagonal paths.

Dijkstra's algorithm was the least performant, primarily due to its lack of a heuristic. The discrepancies in runtime and nodes traversed became increasingly evident as the size of the graphs grew.

The A-Star family of algorithms uses the same underlying architecture as Dijkstra but incorporates admissible heuristics to optimize traversal. Of the three heuristics tested, the Euclidean Distance offered the least optimization. The Manhattan Distance produced significantly higher results, and the Octile Distance performed marginally better than the Manhattan Distance in nearly all cases.

This study could be expanded in several ways, including testing additional algorithms, extending the graphs to three or more dimensions, or restructuring the graphs to incorporate mazes or more complex paths.

5. Appendix

Graph Size	Traversal Type	Edge Connections	Nodes Traversed	Percentage of Graph traversed	Runtime	Averages	% Graph Traversed Averages
100	Diagonal	Uniform	10	10.00%	7.1ms		
100	Orthogonal	Octile	10	10.00%	6.6ms		
100	Diagonal	Random	10	10.00%	1.8ms		
100	Orthogonal	Uniform	10	10.00%	1.9ms		
100	Diagonal	Octile	10	10.00%	7.2ms		
100	Orthogonal	Random	10	10.00%	1.8ms	**4.4ms**	**10.00%**
10000	Diagonal	Uniform	100	1.00%	3.1ms		
10000	Orthogonal	Octile	100	1.00%	9.2ms		
10000	Diagonal	Random	191	1.91%	12.3ms		
10000	Orthogonal	Uniform	100	1.00%	3.4ms		
10000	Diagonal	Octile	100	1.00%	4.8ms		
10000	Orthogonal	Random	100	1.00%	4.4ms	**6.2ms**	**1.15%**
1000000	Diagonal	Uniform	1000	0.10%	28.9ms		
1000000	Orthogonal	Octile	1000	0.10%	26.4ms		
1000000	Diagonal	Random	4022	0.40%	40.0ms		
1000000	Orthogonal	Uniform	1000	0.10%	35.0ms		
1000000	Diagonal	Octile	1000	0.10%	35.0ms		
1000000	Orthogonal	Random	2107	0.21%	50.3ms	**35.9ms**	**0.17%**

Figure 3. A* With Octile Results

Graph Size	Traversal Type	Edge Connections	Nodes Traversed	Percentage of Graph traversed	Runtime	Averages	% Graph Traversed Averages
100	Diagonal	Uniform	82	82.00%	9.2ms		
100	Orthogonal	Octile	58	58.00%	8.0ms		
100	Diagonal	Random	99	99.00%	9.4ms		
100	Orthogonal	Uniform	78	78.00%	5.7ms		
100	Diagonal	Octile	100	100.00%	8.8ms		
100	Orthogonal	Random	53	53.00%	8.1ms	**8.2ms**	**78.33%**
10000	Diagonal	Uniform	8902	89.02%	46.8ms		
10000	Orthogonal	Octile	6149	61.49%	45.9ms		
10000	Diagonal	Random	10000	100.00%	54.6ms		
10000	Orthogonal	Uniform	8445	84.45%	31.1ms		
10000	Diagonal	Octile	10000	100.00%	55.2ms		
10000	Orthogonal	Random	8445	84.45%	43.4ms	**46.2ms**	**86.57%**
1000000	Diagonal	Uniform	898592	89.86%	6,114.3ms		
1000000	Orthogonal	Octile	617865	61.79%	3,950.9ms		
1000000	Diagonal	Random	999999	100.00%	8,655.9ms		
1000000	Orthogonal	Uniform	898592	89.86%	6,490.5ms		
1000000	Diagonal	Octile	898192	89.82%	7,326.5ms		
1000000	Orthogonal	Random	745594	74.56%	4,797.9ms	**6,222.7ms**	**84.31%**

Figure 4. A* With Euclidean Results

Graph Size	Traversal Type	Edge Connections	Nodes Traversed	Percentage of Graph traversed	Runtime	Averages	% Graph Traversed Averages
100	Diagonal	Uniform	10	10.00%	1.7ms		
100	Orthogonal	Octile	10	10.00%	1.7ms		
100	Diagonal	Random	10	10.00%	6.6ms		
100	Orthogonal	Uniform	10	10.00%	2.2ms		
100	Diagonal	Octile	10	10.00%	1.8ms		
100	Orthogonal	Random	10	10.00%	6.5ms	**3.4ms**	**10.00%**
10000	Diagonal	Uniform	100	1.00%	8.9ms		
10000	Orthogonal	Octile	100	1.00%	3.5ms		
10000	Diagonal	Random	100	1.00%	2.4ms		
10000	Orthogonal	Uniform	100	1.00%	2.5ms		
10000	Diagonal	Octile	100	1.00%	5.2ms		
10000	Orthogonal	Random	125	1.25%	6.3ms	**4.8ms**	**1.04%**
1000000	Diagonal	Uniform	1000	0.10%	30.5ms		
1000000	Orthogonal	Octile	1000	0.10%	24.9ms		
1000000	Diagonal	Random	1627	0.16%	33.7ms		
1000000	Orthogonal	Uniform	1000	0.10%	27.0ms		
1000000	Diagonal	Octile	1000	0.10%	29.7ms		
1000000	Orthogonal	Random	2318	0.23%	2.0ms	**24.6ms**	**0.13%**

FIGURE 5. A* With Manhattan Results

Graph Size	Traversal Type	Edge Connections	Nodes Traversed	Percentage of Graph traversed	Runtime	Averages	% Graph Traversed Averages
100	Diagonal	Uniform	10	10.00%	7.1ms		
100	Orthogonal	Octile	10	10.00%	6.6ms		
100	Diagonal	Random	10	10.00%	1.8ms		
100	Orthogonal	Uniform	10	10.00%	1.9ms		
100	Diagonal	Octile	10	10.00%	7.2ms		
100	Orthogonal	Random	10	10.00%	1.8ms	**4.4ms**	**10.00%**
10000	Diagonal	Uniform	100	1.00%	3.1ms		
10000	Orthogonal	Octile	100	1.00%	9.2ms		
10000	Diagonal	Random	191	1.91%	12.3ms		
10000	Orthogonal	Uniform	100	1.00%	3.4ms		
10000	Diagonal	Octile	100	1.00%	4.8ms		
10000	Orthogonal	Random	100	1.00%	4.4ms	**6.2ms**	**1.15%**
1000000	Diagonal	Uniform	1000	0.10%	28.9ms		
1000000	Orthogonal	Octile	1000	0.10%	26.4ms		
1000000	Diagonal	Random	4022	0.40%	40.0ms		
1000000	Orthogonal	Uniform	1000	0.10%	35.0ms		
1000000	Diagonal	Octile	1000	0.10%	35.0ms		
1000000	Orthogonal	Random	2107	0.21%	50.3ms	**35.9ms**	**0.17%**

FIGURE 6. A* With Octile Results

References

[1] Dijkstra, E. W. A note on two problems in connexion with graphs. In *Edsger Wybe Dijkstra: His Life, Work, and Legacy*. Springer, 2022, pp. 287–290.

[2] Hart, P. E., Nilsson, N. J., and Raphael, B. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics 4*, 2 (1968), 100–107.

[3] Javaid, A. Understanding dijkstra's algorithm. *Available at SSRN 2340905* (2013), 14–17.

[4] Sharma, S. K., and Kumar, S. Comparative analysis of manhattan and euclidean distance metrics using a* algorithm. *J. Res. Eng. Appl. Sci 1*, 4 (2016), 196–198.

Department of Computer Science, University of Southern Maine, Portland, ME
Email address: colby.wirth@maine.edu